

Adaptive Explanation Obfuscation via Reinforcement Learning

Gamze Erdoğan
Istanbul Technical University
Istanbul, Türkiye
gamze.erdogan@itu.edu.tr

Betül Güvenç-Paltun
Ericsson
Istanbul, Türkiye
betul.guvenc.paltun@ericsson.com

Leyli Karaçay
Ericsson
Istanbul, Türkiye
leyli.karacay@ericsson.com

Enver Ozdemir
Istanbul Technical University
Istanbul, Türkiye
ozdemiren@itu.edu.tr

Abstract—Artificial intelligence (AI) models typically operate as opaque black boxes, producing outputs without revealing the reasoning behind them. This lack of transparency is undesirable, as clients submitting queries often require insight into how results are obtained. Explainable AI (XAI) addresses this need by providing interpretable explanations of model behavior. However, the very process of explanation can expose vulnerabilities: a malicious client may exploit explanations to extract sensitive information about the underlying model. Such model extraction attacks pose a significant threat to model owners, jeopardizing intellectual property and research investments. Consequently, developing robust defences against these attacks has become a critical area of research and in this work, we present solution to remedy such threats. The solution introduces a reinforcement learning (RL) agent that functions as a protective layer around an XAI model, dynamically adjusting the level of explanation provided based on ongoing interactions. The problem is structured as a continuous adversarial min-max game: an adversary attempts to reconstruct a surrogate model using the explanations given, while the RL agent actively obfuscates the explanations to prevent successful extraction. The system aims to satisfy legitimate user needs while penalizing the agent if the adversary successfully predicts or reconstructs the model. Over time, the agent learns to adapt its strategy, minimizing risks while maintaining the semantic usefulness of explanations for authorized users across varying model architectures.

Index Terms—Explainable AI (XAI), Reinforcement Learning (RL), Model Extraction Attacks, Privacy, Obfuscation.

I. Introduction

Explainable Artificial Intelligence (XAI) focuses on making complex models and their predictions interpretable, providing insights into the reasoning processes behind machine learning outputs. However, while offering explanations assists legitimate users in comprehending model behavior, it simultaneously creates potential vulnerabilities that adversaries may leverage to replicate models.

A model extraction attack occurs when an adversary attempts to replicate a black box model. If an attack utilizes explanations, it is called an explanation-aware model extraction attack (XaMEA). In such attacks, the adversary sends a massive volume of crafted queries to

reveal information about the target model’s behavior. The explanations consist of additional information such as feature importance or confidence scores, which the attacker uses to match both the outputs and explanations of a surrogate model to those of the target model. Therefore, when accompanied by explainability, even highly complex models remain vulnerable to extraction attacks.

To address these challenges, we introduce an approach that integrates a reinforcement learning (RL) agent with XAI to dynamically enhance privacy and security without strictly compromising the explanation quality for regular users.

II. Related Work

To contextualize our dynamic obfuscation approach, we review the literature across four key domains: model extraction vulnerabilities, the privacy-explainability trade-off, static defensive mechanisms, and the application of reinforcement learning in adversarial defense.

A. Vulnerabilities in Machine Learning (Model Extraction)

Standard machine learning models exposed via inference APIs are highly susceptible to model extraction (or model stealing) attacks [1], [2]. In these scenarios, adversaries systematically query the prediction API and utilize the resulting input-output pairs to train a functionally equivalent surrogate model [3]. Recent advances have demonstrated that these attacks can achieve both high accuracy and high fidelity representations of the target model [4]. This vulnerability extends beyond simple tabular classifiers, threatening complex sequential recommenders [5] and large language models [6]. Comprehensive surveys of the field emphasize that modern extraction methodologies are becoming increasingly query-efficient, drastically reducing the time and cost required for an adversary to steal proprietary intellectual property [7].

B. The Privacy-Explainability Trade-off and XaMEA

The deployment of XAI methodologies, such as SHAP [8], LIME [9], and Integrated Gradients [10], has become a standard practice for establishing algorithmic transparency. However, the literature increasingly recognizes a strict privacy-explainability trade-off [11]. Explanations inherently leak gradient information and expose local decision boundaries [12]. Milli et al. [13] theoretically and empirically demonstrated that gradient-based explanations allow for rapid, query-efficient model reconstruction.

This inherent vulnerability spawned the development of Explanation-Aware Model Extraction Attacks (XaMEA) [14]. By exploiting feature importances alongside standard outputs, attackers can accelerate the extraction process. Attack frameworks such as AUTOLYCUS [15] leverage model-agnostic tools like LIME and SHAP to infer decision boundaries with minimal queries, while similar architectures like Megrez [16] effectively target Deep Neural Networks. Furthermore, Shokri et al. [17] established that detailed explanations also exacerbate membership inference risks, confirming that transparency tools uniquely expose models to adversarial exploitation and manipulation [18].

C. Current Defensive Strategies and Their Limitations

Mitigating model extraction traditionally relies on static defense mechanisms. Common techniques include Top- k truncation [5], which limits the informational bandwidth by restricting output dimensions, and prediction generalization, which obscures outputs through deliberate rounding [1]. More proactive defenses attempt to inject controlled, randomized noise into the API predictions [11], [19], often drawing upon differential privacy (DP) frameworks [20] to provide mathematical privacy guarantees for interpretable models [21].

Other state-of-the-art approaches, such as PRADA [22] and adaptive misinformation frameworks [23], [24], analyze incoming query distributions to detect anomalous behavior and intentionally mislead attackers. However, when these static noise or truncation defenses are applied to XAI outputs, they inflict severe utility loss [11]. Static defenses cannot dynamically balance the conflicting needs of maintaining high explanation fidelity for benign users while securing the model against active adversaries.

D. Reinforcement Learning in Adversarial Defense

To effectively combat adaptive adversaries, defensive mechanisms must also be adaptive and unpredictable [25]. Reinforcement Learning (RL) has emerged as a powerful paradigm for addressing complex, dynamic challenges in cybersecurity [26]. Foundational algorithms like Deep Q-Networks (DQN) [27] and Proximal Policy Optimization (PPO) [28] have been successfully adapted for real-time threat detection and network intrusion mitigation [29].

In the specific context of adversarial machine learning, frameworks like Robust Adversarial Reinforcement Learn-

ing (RARL) [30] formulate security as a continuous min-max game. This forces the RL agent to learn robust behavioral policies against active perturbations. By extending this game-theoretic RL approach to XAI obfuscation, our system overcomes the core limitations of static privacy mechanisms, allowing the model to dynamically optimize the trade-off between interpretability and security on a per-interaction basis.

III. Proposed System Architecture

The system consists of an inference provider who manages the ML model along with its associated explainability module. The RL agent acts as a protective mechanism that decides how much information to conceal or modify in an explanation.

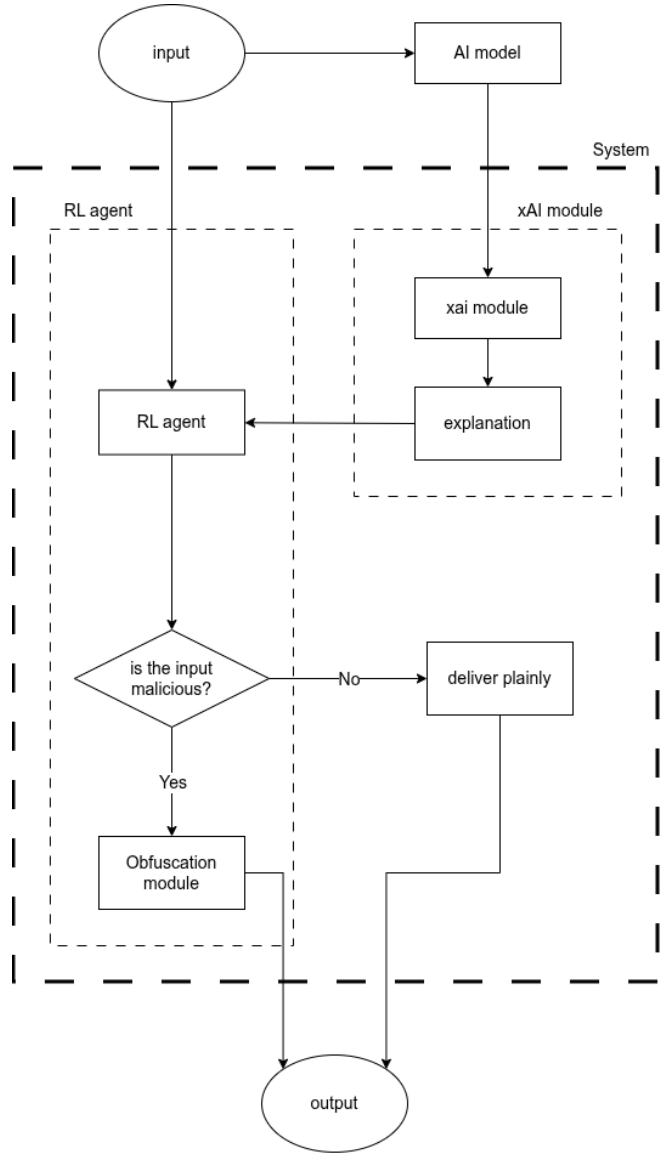


Fig. 1. High-level overview of the system architecture.

As illustrated in Fig. 1, the high-level components of the system are:

- AI Model: A model hosted and served by the inference provider. In our evaluations, we utilize both Decision Tree Ensembles (XGBoost) and Deep Neural Networks (DNN) to demonstrate architectural generalizability.
- Input: The query submitted for processing.
- RL agent: Determines how much information to conceal in an explanation to limit model extraction while preserving utility. We utilize a Proximal Policy Optimization (PPO) algorithm for this agent.
- Obfuscation module: Intentionally obfuscates explanations to prevent attackers from learning too much about the model.
- XAI module: Generates baseline explanations for the outputs of the model using frameworks such as SHAP or LIME.

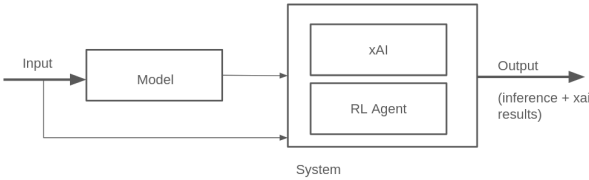


Fig. 2. System workflow: inference model output is explained by the XAI module and selectively obfuscated by the RL agent before returning results.

The workflow operates as follows (see Fig. 2):

- 1) An input query is submitted to the system.
- 2) The input is processed on the model, generating an output.
- 3) The input is simultaneously sent to the RL agent for analysis.
- 4) The RL agent examines the input’s relation to historical queries (through the rolling-window state of Eq. (1)) and determines the degree of obfuscation required.
- 5) The XAI module receives the output and generates a baseline explanation, which is sent to the RL agent.
- 6) The RL agent decides whether to apply obfuscation to the explanation via the obfuscation module.
- 7) The processed output and the (potentially obfuscated) explanation are returned to the user.

IV. Game Design for the RL Agent

A. Reinforcement Learning Configuration

Algorithm Selection: To drive the dynamic obfuscation module, we employ the Proximal Policy Optimization (PPO) algorithm [28]. PPO was selected over alternative reinforcement learning algorithms, such as Deep Q-Networks (DQN) or Soft Actor-Critic (SAC), due to its inherent stability and compatibility with continuous action spaces. Because our obfuscation intensity a_t is a continuous variable defined in the range $[0, 1]$, value-based methods like DQN, which require discrete action spaces,

are unsuitable without arbitrary discretization. While SAC accommodates continuous actions, PPO’s clipped surrogate objective function prevents destructively large policy updates, ensuring stable and monotonic improvement during the delicate min-max adversarial training process.

State-Space Formulation and Network Architecture: The RL agent must make contextual decisions based on the current query and the history of adversarial interactions. The full history of a session (Eq. (2) in Section IV-B) grows with every query, whereas an MLP policy requires an input of fixed size. We therefore encode the history in fixed size as a rolling window of the user’s last k queries. Each query is first standardised with the training-set statistics, $z_t = (x_t - \bar{x})/\sigma_x$, and the state is the concatenation of the k most recent standardised queries and the normalized step counter:

$$s_t = \left[z_{t-k+1} \parallel \dots \parallel z_{t-1} \parallel z_t \parallel \frac{t}{T_{max}} \right] \quad (1)$$

Where t is the current query step and T_{max} represents the maximum allowed queries in a session. At the start of a session ($t < k$), missing slots are filled with zeros (the feature mean). The size of s_t , $kd + 1$ for d features, is independent of t . The window preserves the order and recency of the user’s queries, so the policy can condition on local query patterns such as bursts of near-duplicate or slightly perturbed queries, which session-wide averages would smooth out. We use $k = 8$. This combined vector is processed by the PPO agent, which is parameterized using a standard Multi-Layer Perceptron (MLP) architecture. The MLP network acts as both the actor, outputting the continuous action a_t , and the critic, estimating the state-value function to guide policy updates.

Convergence Behavior: Each training episode is one simulated session of $T_{max} = 1000$ queries, drawn in random order from a pool of 5,000 training samples; a new random session is drawn for every episode, so the agent cannot memorise a fixed query sequence. Operating over 50,000 environmental timesteps (50 episodes) with a learning rate of 3×10^{-4} , the agent learns a stable defensive policy. Empirical learning curves derived from the environment’s rolling episode rewards indicate that the agent steadily maximized its reward function, confirming that PPO effectively managed the volatile signals generated by the adversary’s simultaneously evolving surrogate model. Performance is not yet saturated at this budget: training for 500,000 timesteps further improves the security of the learned policies (Section V-C).

The RL agent is trained using a game-based framework designed to enhance model security while maintaining explanation utility. The setup simulates a continuous adversarial scenario, as depicted in Fig. 3.

B. Min-Max Game Formulation

Step 1: State at time t

The state s_t at time t is the history of queries received

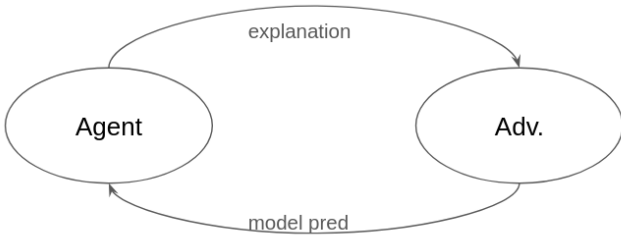


Fig. 3. Game-based training loop: the RL agent obfuscates explanations; the adversary trains a surrogate model, and rewards/penalties are assigned based on security and explanation utility.

from a specific user, where x_i is the query and y_i is the output.

$$s_i = \{(x_1, y_1), (x_2, y_2), \dots, (x_i, y_i)\} \quad (2)$$

Eq. (2) defines the information available to the defender; since it grows at every step, the agent observes it through the fixed-size encoding of Eq. (1). The outputs $y_i = M(x_i)$ are a deterministic function of the queries and add no information, so the encoding keeps the queries only.

Step 2: RL Agent Action

The RL agent determines an obfuscation function f_{obfus} to apply to the true explanation E_{true} . The action a_t is a continuous variable $a_t \in [0, 1]$ representing the intensity of obfuscation. The applied transformation is defined as:

$$E_{output} = (1 - a_t)E_{true} + a_t \cdot \epsilon \quad (3)$$

Where ϵ represents Gaussian noise scaled by the standard deviation of the true explanations ($\epsilon \sim \mathcal{N}(0, \sigma_E^2)$). If $a_t = 0$, $E_{output} = E_{true}$ (full explanation). If $a_t = 1$, E_{output} relies entirely on the generated noise (full obfuscation).

Step 3: Adversary Action

The adversary leverages E_{output} to construct a surrogate model S targeting the original model M . The adversary minimizes the extraction loss $L_{extract}$:

$$\min L_{extract}(S(x), M(x)|E_{output}) \quad (4)$$

The surrogate is trained incrementally: it is re-initialised at the start of every episode and updated with one gradient step per query on the input $[x_t, E_{output}]$ and label $M(x_t)$. $L_{extract}$ is the surrogate’s cross-entropy with the target’s labels on its $W = 32$ most recent queries, measured before the update on x_t .

Step 4: Reward Function

The reward function R_t balances security (maximizing adversary loss) and utility (minimizing explanation deviation).

$$R_t = \lambda \cdot L_{extract}(S, M) - \mu \cdot \frac{\|E_{true} - E_{output}\|}{\|E_{true}\|} \quad (5)$$

Where λ and μ are non-negative scalar hyperparameters weighing the importance of model security and explanation fidelity, respectively. The distortion term is normalized by $\|E_{true}\|$, as in the evaluation metric of Section V-B, so that it is on the same scale as $L_{extract}$.

V. Experimental Evaluation

A. Experimental Setup

To evaluate the proposed methodology and demonstrate its structural generalizability, we implemented two distinct machine learning pipelines:

- 1) Adult Income (XGBoost + SHAP): A black-box XGBoost target model providing interpretability via SHAP TreeExplainer.
- 2) Credit Card Fraud (DNN + LIME): A Deep Neural Network target model providing interpretability via LIME (Local Interpretable Model-agnostic Explanations).

In both environments, the adversary was designed as an adaptive attacker aiming to replicate the target model using a surrogate Multi-Layer Perceptron (MLP) Classifier configured with two hidden layers of 64 and 32 neurons. The RL defending agent was trained independently for each pipeline using the PPO algorithm over 50,000 timesteps.

Attackers. The natural-query attacker queries natural data in random order, as during training. The synthetic-query attacker owns only 10 seed samples and generates new queries by perturbing earlier ones with random-sign steps of $0.1\sigma_x$ on a random subset of features, in the spirit of the augmentation-based attacks that PRADA [22] is designed to detect. All strategies are evaluated on the same three sessions of 500 held-out queries per attacker.

Agents. Following the formulation of Section IV-B, we vary both weights: $\lambda \in \{0, 0.1, 0.25, 0.5, 0.75, 1, 1.5, 2, 3, 5\}$ at each $\mu \in \{0, 0.01, 0.05, 0.1, 0.2, 0.5\}$ (except $\lambda = \mu = 0$). Each agent is trained with the rolling-window state of Eq. (1) and, as an ablation, with the history-free state $s_t = [z_t \parallel t/T_{max}]$, over three random seeds; reported values are averages over the seeds.

B. Evaluation Metrics

We benchmark the defensive policies across three primary metrics to assess the balance between adversarial security and user interpretability:

1. Average Adversary Error ($L_{extract}$): Measures the cross-entropy loss of the adversary’s surrogate model. Higher values indicate successful defense against extraction. (We rename this metric from “extraction loss” because the defender maximizes it.)

2. Average Explanation Distortion (L2 Distortion): The normalized L2 norm between the true explanation and the obfuscated output, representing the raw geometric distortion.

3. Semantic Interpretability Metrics: To ensure the obfuscation policy does not destroy the inherent meaning of the explanations, we evaluate two semantic utility metrics:

- Spearman’s Rank Correlation (ρ): Measures the monotonic relationship between the feature importance rankings of the original explanation E_{true} and

the obfuscated explanation E_{output} . A higher ρ indicates that the relative hierarchy of feature importance is preserved.

- **Top- k Feature Agreement:** Calculates the intersection of the k most critical features before and after obfuscation, defined as $\frac{|S_{true} \cap S_{output}|}{k}$, where S represents the set of indices for the top k features. We report results for $k = 3$.

C. Results and Baselines

The trained PPO agents were benchmarked against multiple static defense strategies prevalent in the literature: Top- k truncation ($k = 3$), Gaussian Noise addition (level=0.5), Precision Reduction (2 decimals), and Random Subset omission ($p = 0.5$). We also compare against the state-of-the-art detector PRADA [22], implemented from its published algorithm on standardised features. As PRADA detects rather than obfuscates, we use it as a gate: explanations are exact until the client is flagged and fully obfuscated ($a_t = 1$) afterwards. Its threshold δ is the largest value that raises no alarm on benign sessions.

Correction. In the original submission, the adversary error was set to a constant 1.0 whenever the adversary’s 32-query window contained a single class, which is almost always the case on the highly imbalanced Credit Card Fraud data; this produced its uniform values of 0.98–0.99. All results in this version are recomputed with the corrected metric.

As demonstrated in Table I, static methods struggle to balance the tradeoff effectively on the SHAP pipeline. While Gaussian Noise and Random Subset methods exhibit higher security, they inflict severe utility loss (1.0688 and 0.5883, respectively), rendering the explanations functionally useless for benign users. In contrast, the RL Agent ($\mu = 0.05$) successfully increases security (0.2717) over the baseline (0.2345, +16%) while maintaining an exceptional explanation fidelity (Top-3 Agreement ≈ 0.97). It thus nearly matches the security of Random Subset (0.2758) at one seventh of its distortion, and is more secure than Gaussian Noise at less than a tenth of its distortion. Fig. 4 shows the complete trade-off for both datasets.

Varying λ . Fig. 5 reports all four metrics as a function of the security weight λ . On Adult Income, increasing λ raises the adversary error from the undefended 0.2345 to about 0.28 while distortion grows and the rank metrics decrease, i.e. λ moves the agent along the security–utility trade-off. Since multiplying R_t by a positive constant does not change the optimal policy, the trade-off depends on the ratio λ/μ , which our grid confirms: for example, the no-history agents with $(\lambda, \mu) = (0.25, 0.01)$, $(1, 0.05)$ and $(2, 0.1)$, all with $\lambda/\mu = 20$, reach adversary errors of 0.274, 0.272 and 0.270 at distortions of 0.09, 0.08 and 0.09.

Comparison with PRADA. PRADA never flags the natural-query attacker, whose queries are, by construction, indistinguishable from those of a benign client (Tables I and II). Against the synthetic-query attacker it is effective

but coarse (Table III): it flags the client at query 101–102, the earliest point its statistical test allows, and fully obfuscates everything afterwards. On Adult Income, the RL agent ($\lambda = 1$, $\mu = 0.01$, no history) comes close to PRADA’s security (0.1898 vs. 0.2101) at one sixth of its distortion (0.24 vs. 1.49), while preserving the feature ranking ($\rho = 0.85$ vs. 0.20). The two defenses are complementary. Unlike detection-based defenses such as PRADA, which cannot identify attackers that query natural data, the RL agent protects against both attacker types. Against synthetic-query attackers, PRADA attains the highest security but destroys the explanations once triggered, whereas the RL agent attains comparable security at a fraction of the distortion. PRADA, in turn, releases exact explanations to clients it does not flag, whereas the RL agent studied here obfuscates for every client (see below).

Effect of the query history. Across both datasets and attackers, the rolling-window agent does not reach a better trade-off than the history-free ablation: at equal distortion, the no-history agent is at least as secure (Fig. 4). This is expected in the game studied here: every training session is an extraction attempt with independently drawn queries, so earlier queries carry no information about the current one, while the larger input makes the policy harder to learn. This is not an artefact of the training budget: at 500,000 timesteps ($\lambda = 1$, $\mu = 0.05$), the no-history agent reaches an adversary error of 0.3015 (+29% over no defense) at a distortion of 0.14 ($\rho = 0.88$, Top-3 agreement 0.95) on Adult Income, and 0.1991 at a distortion of 0.19 against the synthetic-query attacker, whereas the rolling-window agent reaches 0.2671 at a distortion of 0.33. The history encoding becomes relevant when benign and malicious clients are served by the same agent, so that past queries indicate which kind of client is being served; we leave this setting to future work. Relatedly, as the agent is trained only on extraction sessions, it applies the same obfuscation to benign clients, who therefore bear the distortion reported above; training on mixed sessions, rewarding exact explanations for benign clients, is the natural remedy.

The evaluation on the Credit Card Fraud dataset (Table II) further demonstrates the severe limitations of static defenses when applied to LIME explanations. While Top- k Truncation ($k = 3$) maintains perfect Top-3 Agreement, it completely disrupts the overall feature ranking ($\rho \approx 0.53$). Conversely, Precision Reduction preserves general ranking ($\rho \approx 0.77$) but loses over half of the most critical top-3 features (Agreement ≈ 0.44). Our adaptive policy ($\lambda = 1.0, \mu = 0.05$, no history) maintains high rank correlation ($\rho \approx 0.97$) and preserves the most critical features (Top-3 Agreement ≈ 0.96). However, on this dataset no defense, static or adaptive, yields a meaningful security gain: the adversary error stays between 0.081 and 0.093 for every strategy. Fraud accounts for only 0.17% of the transactions, so sessions of natural queries contain almost no positive cases and a surrogate that predicts

TABLE I
Security vs. Distortion Trade-off: Adult Income Dataset (XGBoost + SHAP), [natural-query attacker](#)

Strategy	Adversary Error $\uparrow$	Expl. Distortion $\downarrow$	Spearman's ρ $\uparrow$	Top-3 Agreement $\uparrow$
Static Baselines				
No Defense	0.2345	0.0000	1.0000	1.0000
Precision Reduction (2 dec)	0.2343	0.0062	0.9982	0.9949
Top- k ($k = 3$)	0.2438	0.1474	0.9411	1.0000
Gaussian Noise (lvl=0.5)	0.2521	1.0688	0.5901	0.7211
Random Subset (p=0.5)	0.2758	0.5883	0.5575	0.5471
PRADA [22]	0.2345	0.0000	1.0000	1.0000
RL Adaptive Agents ($\lambda = 1.0$), rolling-window state				
RL ($\mu = 0.1$)	0.2536	0.1607	0.9128	0.9447
RL ($\mu = 0.05$)	0.2651	0.3763	0.8140	0.8977
RL ($\mu = 0.01$)	0.2795	0.6804	0.6876	0.8396
RL ($\mu = 0.0$)	0.2826	0.7535	0.6588	0.8264
RL Adaptive Agents ($\lambda = 1.0$), no history (ablation)				
RL ($\mu = 0.1$)	0.2552	0.0529	0.9613	0.9793
RL ($\mu = 0.05$)	0.2717	0.0813	0.9351	0.9704
RL ($\mu = 0.01$)	0.2912	0.4182	0.7567	0.8613
RL ($\mu = 0.0$)	0.3007	0.9864	0.5178	0.7407

TABLE II
Security vs. Distortion Trade-off: Credit Card Fraud Dataset (DNN + LIME), [natural-query attacker](#)

Strategy	Adversary Error $\uparrow$	Expl. Distortion $\downarrow$	Spearman's ρ $\uparrow$	Top-3 Agreement $\uparrow$
Static Baselines				
No Defense	0.0866	0.0000	1.0000	1.0000
Precision Reduction (2 dec)	0.0851	0.5643	0.7728	0.4438
Top- k ($k = 3$)	0.0822	0.7632	0.5287	1.0000
Gaussian Noise (lvl=0.5)	0.0812	0.7064	0.7967	0.5256
Random Subset (p=0.5)	0.0933	0.6977	0.6469	0.4987
PRADA [22]	0.0866	0.0000	1.0000	1.0000
RL Adaptive Agents ($\lambda = 1.0$), rolling-window state				
RL ($\mu = 0.1$)	0.0863	0.3000	0.8524	0.8082
RL ($\mu = 0.05$)	0.0859	0.4240	0.7856	0.7401
RL ($\mu = 0.01$)	0.0848	0.5319	0.7295	0.6859
RL ($\mu = 0.0$)	0.0847	0.5440	0.7231	0.6788
RL Adaptive Agents ($\lambda = 1.0$), no history (ablation)				
RL ($\mu = 0.1$)	0.0864	0.0250	0.9913	0.9840
RL ($\mu = 0.05$)	0.0864	0.0674	0.9739	0.9550
RL ($\mu = 0.01$)	0.0869	0.2622	0.8806	0.8186
RL ($\mu = 0.0$)	0.0870	0.3842	0.8186	0.7484

TABLE III
Synthetic-Query Attacker: Adversary Error ($\uparrow$) / Explanation Distortion ($\downarrow$)

Strategy	Adult Income	Credit Card
No Defense	0.1259 / 0.00	0.0627 / 0.00
Random Subset (p=0.5)	0.1808 / 0.59	0.0685 / 0.70
PRADA [22]	0.2101 / 1.49	0.0623 / 1.44
RL ($\lambda = 1$), rolling-window state		
RL ($\mu = 0.05$)	0.1530 / 0.24	0.0633 / 0.30
RL ($\mu = 0.01$)	0.1742 / 0.55	0.0638 / 0.38
RL ($\lambda = 1$), no history (ablation)		
RL ($\mu = 0.05$)	0.1567 / 0.08	0.0629 / 0.12
RL ($\mu = 0.01$)	0.1898 / 0.24	0.0634 / 0.37

the majority class imitates the target well regardless of the explanations. The RL agents learn that obfuscation

does not pay here and keep the distortion low. Defending explanations of models trained on highly imbalanced data therefore requires a threat model with class-balanced queries, which we leave to future work.

VI. Discussion and Practical Deployment

A. Addressing Computational Overhead

A primary concern when integrating adaptive defense mechanisms into production machine learning pipelines is the introduction of unacceptable latency during real-time inference. Traditional reinforcement learning training is computationally intensive; however, the deployment phase of our proposed architecture introduces [little](#) overhead.

The trained PPO actor is parameterized as a lightweight Multi-Layer Perceptron (MLP). During inference, calculating the obfuscation intensity a_t requires only a single

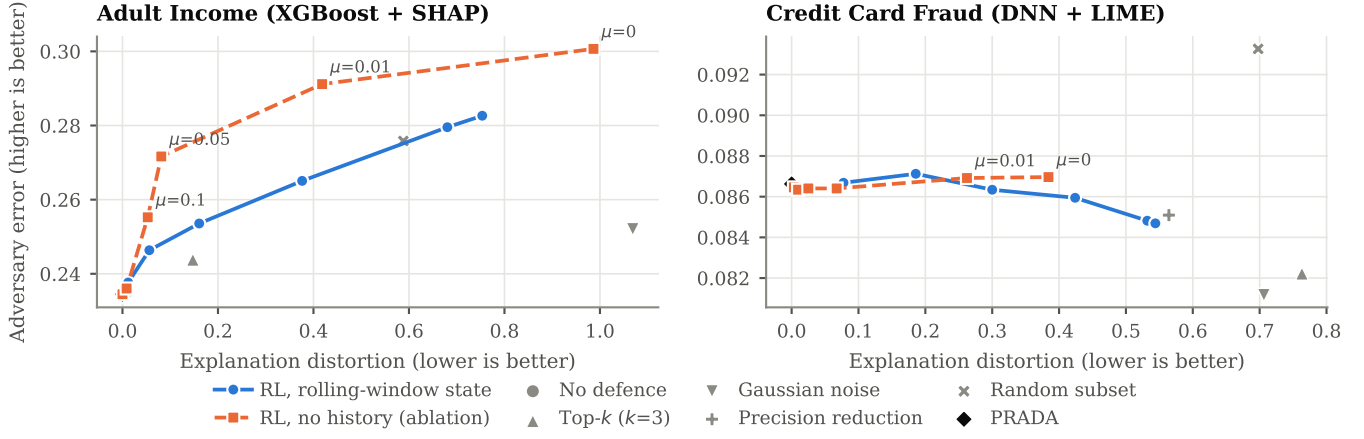


Fig. 4. Security versus Distortion trade-off for both datasets (natural-query attacker). RL agents use $\lambda = 1$ with μ varied, averaged over three seeds; static baselines and PRADA are shown for reference.

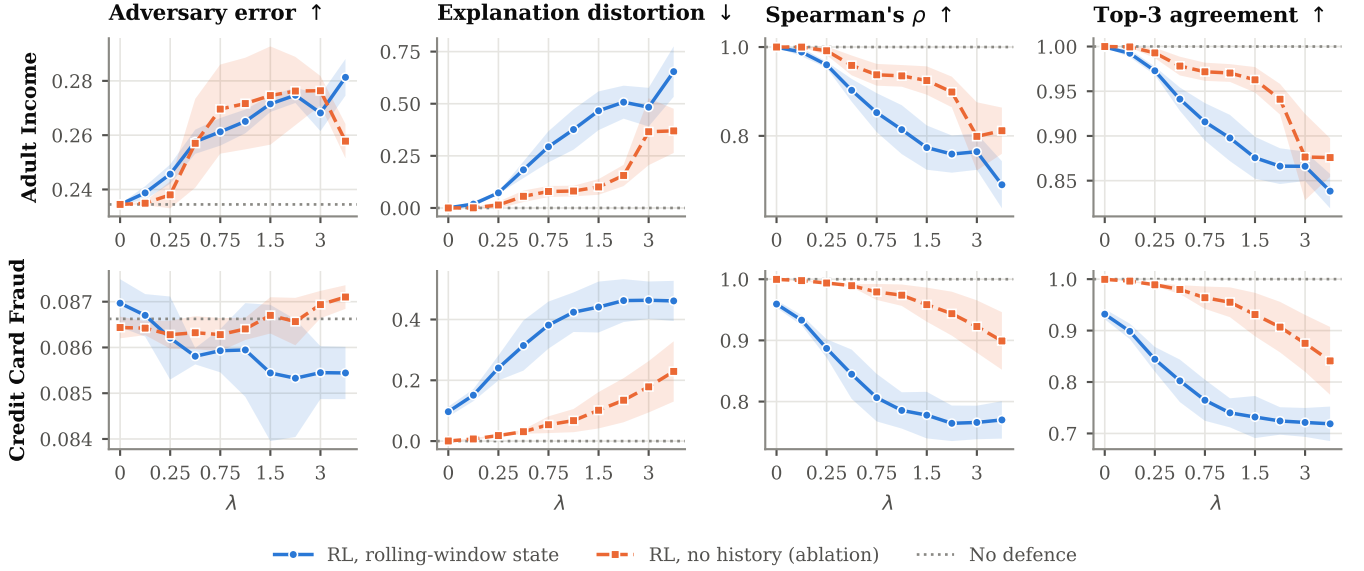


Fig. 5. All four metrics as a function of the security weight λ ($\mu = 0.05$, natural-query attacker), for the rolling-window state and the no-history ablation; mean $\pm$ standard deviation over three seeds. Note the narrow adversary-error range on Credit Card Fraud.

forward pass through this shallow neural network. We measured the per-query cost on a single CPU core (median over all evaluation queries; Table IV). The RL defense (state encoding, policy forward pass and obfuscation) takes 0.34 ms, less than PRADA. It adds only 2.3% to LIME explanations; with the much faster TreeSHAP it adds 56%, yet a query is still served in about 1.2 ms. Therefore, the dynamic obfuscation layer can be deployed as an API gateway proxy without violating strict latency service-level agreements (SLAs).

B. Dynamic Threat Posturing

The ablation study demonstrates that a single RL architecture can accommodate diverse security postures.

In a practical deployment, a centralized security orchestration system could dynamically swap the active RL agent weights based on the current global threat level. During periods of normal operation, the system can utilize a high-utility agent ($\mu = 0.05$). If an intrusion detection system (IDS) flags an ongoing distributed XaMEA attack, the gateway can instantly pivot to a high-security agent ($\mu = 0.01$), actively stifling the extraction attempt while still providing baseline, albeit noisy, interpretability.

VII. Advantages of the Proposed Solution

The dynamic integration of the RL agent provides several technical benefits:

TABLE IV

Time per Query (ms, median, single CPU core); in parentheses, the defense's cost relative to explanation generation

Component	Adult (SHAP)	Credit (LIME)
Serving the query		
Model prediction	0.30	0.20
Explanation	0.61	15.09
Defense		
No Defense	0.002 (0.4%)	0.002 (0.0%)
Precision Reduction (2 dec)	0.008 (1.4%)	0.009 (0.1%)
Random Subset ($p=0.5$)	0.010 (1.7%)	0.010 (0.1%)
Top- k ($k=3$)	0.010 (1.7%)	0.011 (0.1%)
Gaussian Noise ($lvl=0.5$)	0.022 (3.7%)	0.023 (0.2%)
PRADA [22]	0.44 (73%)	0.46 (3.1%)
RL, no history	0.34 (56%)	0.34 (2.3%)
RL, rolling-window state	0.34 (56%)	0.35 (2.3%)

- **Enhanced Security:** Mitigates model extraction attacks by limiting explanation detail during suspicious activity.
- **Adaptive Explanations:** Continuously monitors query patterns and adapts the detail level based on historical interactions.
- **Balanced Utility:** As evidenced by the empirical benchmarks, authorized users receive high-quality insights with negligible semantic distortion, while adversaries are successfully stifled.
- **Robustness:** The RL agent adapts over time and applies unchanged across varied architectures (Tree Ensembles and DNNs) and XAI methods (SHAP and LIME); where obfuscation cannot improve security, as on the highly imbalanced Credit Card Fraud data, it learns to preserve explanation utility instead.

Acknowledgment

The authors acknowledge the use of generative AI tools (specifically, large language models) to assist with code optimization, script debugging, and language proofreading during the preparation of this manuscript. All AI-assisted outputs were thoroughly reviewed, edited, and validated by the human authors, who take full and final responsibility for the originality, accuracy, and integrity of this work.

References

- [1] F. Tramèr, F. Zhang, A. Juels, M. K. Reiter, and T. Ristenpart, "Stealing machine learning models via prediction apis," in Proc. 25th USENIX Security Symp. (USENIX Security 16), Austin, TX, USA, 2016, pp. 601–618.
- [2] N. Papernot, P. McDaniel, I. Goodfellow, S. Jha, Z. B. Celik, and A. Swami, "Practical black-box attacks against machine learning," in Proc. ACM Asia Conf. Comput. Commun. Security (AsiaCCS '17), 2017, pp. 506–519.
- [3] T. Orekondy, B. Schiele, and M. Fritz, "Knockoff nets: Stealing functionality of black-box models," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2019, pp. 4954–4963.
- [4] M. Jagielski, N. Carlini, D. Berthelot, A. Kurakin, and N. Papernot, "High accuracy and high fidelity extraction of neural networks," in Proc. 29th USENIX Security Symp., 2020, pp. 1345–1362.
- [5] Z. Yue, Z. He, H. Zeng, and J. McAuley, "Black-box attacks on sequential recommenders via data-free model extraction," in Proc. 15th ACM Conf. Recommender Syst. (RecSys '21), New York, NY, USA, 2021, pp. 44–54.
- [6] N. Carlini et al., "Extracting training data from large language models," in Proc. 30th USENIX Security Symp., 2021, pp. 2633–2650.
- [7] D. Oliynyk, R. S. M. Goh, and J. W. Ng, "I know what you trained last summer: A survey on stealing machine learning models and defences," ACM Computing Surveys, vol. 55, no. 14, pp. 1–36, 2023.
- [8] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.
- [9] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should i trust you?": Explaining the predictions of any classifier," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 1135–1144.
- [10] M. Sundararajan, A. Taly, and Q. Yan, "Axiomatic attribution for deep networks," in Int. Conf. Machine Learning (ICML), 2017, pp. 3319–3328.
- [11] S. Saifullah, D. Mercier, A. Lucieri, A. Dengel, and S. Ahmed, "The privacy-explainability trade-off: unraveling the impacts of differential privacy and federated learning on attribution methods," Frontiers in Artificial Intelligence, vol. 7, p. 1236947, Jul 2024.
- [12] A.-K. Dombrowski, M. Alber, C. Osendorfer, and M. Kessel, "Explanations can be manipulated and geometry is to blame," in Advances in Neural Information Processing Systems (NeurIPS), vol. 32, 2019.
- [13] S. Milli, L. Schmidt, A. D. Dragan, and M. Hardt, "Model reconstruction from model explanations," in Proc. Conf. Fairness, Accountability, and Transparency (FAT* '19), 2019, pp. 1–9.
- [14] A. Yan, T. Huang, L. Ke, X. Liu, Q. Chen, and C. Dong, "Explanation leaks: Explanation-guided model extraction attacks," Information Sciences, vol. 632, pp. 269–284, 2023.
- [15] A. C. Oksuz, A. Halimi, and E. Ayday, "Autolytus: Exploiting explainable artificial intelligence (xai) for model extraction attacks against interpretable models," Proc. Priv. Enhancing Technol. (PoPETs), vol. 2024, no. 4, pp. 684–699, 2024.
- [16] T. Miura, S. Hasegawa, and T. Shibahara, "Megrez: Explanation-aware model extraction attacks on deep neural networks," IEEE Transactions on Dependable and Secure Computing, 2021.
- [17] R. Shokri, M. Strobel, and Y. Zick, "On the privacy risks of model explanations," in Proc. AAAI/ACM Conf. AI, Ethics, and Society (AIES), 2021, pp. 231–241.
- [18] D. Slack, S. Hilgard, E. Jia, S. Singh, and S. Lakkaraju, "Fooling lime and shap: Adversarial attacks on post hoc explanation methods," in Proc. AAAI/ACM Conf. AI, Ethics, and Society (AIES '20), 2020, pp. 180–186.
- [19] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in Theory of Cryptography Conf. (TCC), 2006, pp. 265–284.
- [20] M. Abadi et al., "Deep learning with differential privacy," in Proc. ACM SIGSAC Conf. Comput. Commun. Security (CCS '16), 2016, pp. 308–318.
- [21] F. Harder, M. Asafi, and M. Schmitt, "Interpretable and differentially private predictions," in Proc. AAAI Conf. Artificial Intelligence, vol. 34, 2020, pp. 4083–4090.
- [22] M. Juuti, S. Szyller, S. Marchal, and N. Asokan, "Prada: Protecting against dnn model stealing attacks," in Proc. IEEE European Symp. Security Privacy (EuroS&P), 2019, pp. 512–527.
- [23] S. Kariyappa and M. K. Qureshi, "Defending against model stealing attacks with adaptive misinformation," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2020, pp. 770–778.
- [24] T. Lee, B. Edwards, I. Molloy, and D. Su, "Defending against machine learning model stealing attacks using deceptive perturbations," in Proc. IEEE Symp. Security Privacy Workshops (SPW), 2019, pp. 33–39.
- [25] L. Huang, A. D. Joseph, B. Nelson, B. I. Rubinstein, and J. D. Tygar, "Adversarial machine learning," in Proc. 4th ACM

- Workshop Security and Artificial Intelligence (AISec '11), 2011, pp. 43–58.
- [26] T. T. Nguyen and V. J. Reddi, “Deep reinforcement learning for cyber security,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 8, pp. 3779–3795, 2021.
 - [27] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
 - [28] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
 - [29] A. Al-Dujaili, A. Huang, E. Hemberg, and U.-M. O’Reilly, “Towards robust applied machine learning in cyber security,” in *Proc. IEEE Security Privacy Workshops (SPW)*, 2018, pp. 248–254.
 - [30] L. Pinto, J. Davidson, R. Sukthankar, and A. Gupta, “Robust adversarial reinforcement learning,” in *Int. Conf. Machine Learning (ICML)*, 2017, pp. 2817–2826.